Objectives

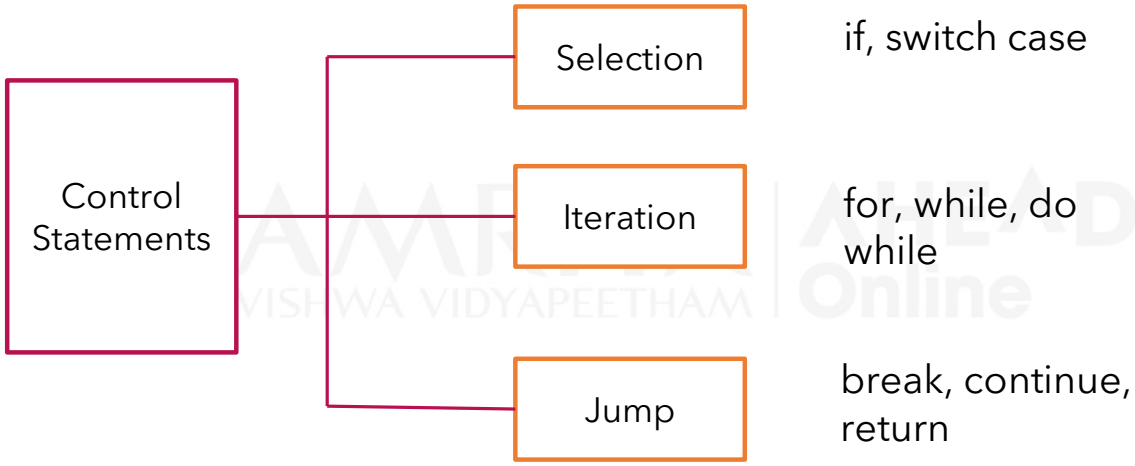
Control Statements in Java

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Control Statements

- ✓ Control statements cause the flow of execution to advance, and branch based on changes to the state of a program.
- ✓ There are three categories of control statements:
  - ✓ Selection
  - ✓ Iteration
  - ✓ Jump

Control Statements



Selection Statements

- ✓ Java supports two selection statements:
  - ✓ if
  - ✓ Switch
- ✓ They control the flow of the program's execution based upon conditions

```
if (condition)
    statement1;
```

if-else-if Ladder

```
if(condition)
    statement;
else if(condition)
    statement;
else if(condition)
    statement;
...
else
    statement;
```

Nested ifs

A *nested* **if** is an **if** statement that is the target of another **if** or **else**.

```
if (i == 10) {
  if (j < 20) a = b;
  if(k > 100) c = d; // this if is
  else a = c; // associated with this else
}
else a = d; // this else refers to if(i == 10)
```

switch statement

- ✓ The switch statement is Java's multiway branch statement.
- ✓ *expression* must be of type byte, short, int, char or String.
- ✓ If you omit the break, execution will continue into the next case.

```
switch (expression) {
  case value1:
     // statement sequence
     break;
  case value2:
     // statement sequence
     break;
.
.
.
  case valueN :
     // statement sequence
     break;
  default:
    // default statement sequence
}
```

switch vs if statement

- ✓ The switch can only test for equality, whereas if can evaluate any type of Boolean expression.
- ✓ Switch looks only for a match between the value of the expression and one of its case constants.
- ✓ No two case constants in the same switch can have identical values.
- ✓ A switch statement is usually more efficient than a set of nested ifs.

Takeaways

- ✓ Control statements.
- ✓ Various selection statements and their syntax.

Reference

[1] Herbert Schildt, *"Java: The Complete Reference, Eleventh Edition"*, *Oracle 2018*